Adding a New Activity to Coggy Flasher

Oct 14, 2026 · @Shreyas Proddutoori

Overview

A new activity needs one catalogue entry in `index.html` and one firmware file in the right folder. Wiring pins are optional. Nothing else in the code changes.

1. **Firmware file**: the compiled `.bin`, saved as `firmware.bin` in the activity's folder under `firmware/`.
2. **Catalogue entry**: a block in the `CATALOGUE` section of `index.html`, which makes the card appear on the shelf.
3. **Wiring sheet** (optional): a block in the `WIRING SHEETS` section, which fills the card's Wiring pop-up.
4. **Upload**: put the changed files on the server.

The flasher finds the firmware from the path in the catalogue entry, so the folder names and the path must match exactly.

File structure

Every activity has its own folder, named with its code, inside its class folder. The build inside is always called `firmware.bin`.

```
coggy-flasher/
├── index.html            ← the whole website, including the catalogue and
wiring lists
├── assets/               ← logos, icons, coggy-drivers.exe
├── vendor/
|   └── esptool-js/bundle.js
└── firmware/
    ├── grade-03/
    |   ├── CS-G03-A01/             ← Universal activity: only T1 is needed
    |   |   └── T1/firmware.bin
    |   └── CS-G03-A02/             ← Table wise activity: one build per
table
    |       ├── T1/firmware.bin
    |       ├── T2/firmware.bin
    |       ├── ...
    |       └── T15/firmware.bin
    ├── grade-04/
    └── ...
```

Table wise code: each table has its own build, in `T1` to `T15`. A table without a build shows a warning, and Flash opens the "Oh no, sorry!" pop-up.

Universal code: one build for every table. Put it in the `T1` folder. The flasher reads `T1` for every table, so the other table folders aren't needed.

Table folders are named by ID, not by name:

Folder	Table	Folder	Table	Folder	Table
T1	Pi	T6	Hawking	T11	Bose
T2	Newton	T7	Curie	T12	Homi
T3	Kalam	T8	Tesla	T13	NA
T4	Euler	T9	Ramanujan	T14	NA
T5	Einstein	T10	Dhawan	T15	NA

The table list lives in `index.html` under `const TABLES`. Rename T13 to T15 there when those tables get names.

Folder and file names are case-sensitive on the server. `CS-G03-A01` and `cs-g03-a01` are different folders, and `Firmware.bin` won't be found.

Activity template

Copy this block into the `"activities": [ ... ]` list of the right class in `index.html`. Search for `THE CATALOGUE` to find the list. Then change the values.

```
{
  "code": "CS-G08-A07",
  "name": "Line Following Robot",
  "category": "Mechatronics",
  "board": "esp32",
  "version": "1.0.0",
  "codeType": "table",
  "wifiDetails": false,
  "ssid": "",
  "password": "",
  "description": "Two IR sensors keep the robot on a black line.",
  "components": [
    "IR sensor x2",
    "L298N",
    "Gear motors x2"
  ],
  "parts": [
    {
      "file": "/firmware/grade-08/CS-G08-A07/{table}/firmware.bin",
      "offset": "0x0"
    }
  ]
},
```

The catalogue is strict JSON, so three rules decide whether the page loads:

- Every block except the last one in a list ends with `},`. The last block ends with `}` and no comma.
- Every text value goes in double quotes. `true` and `false` don't take quotes.
- No comments (`//`) inside the catalogue. Notes go in the comment box above it.

If the shelf shows 0 activities after a change, the JSON has a mistake. Paste the catalogue into jsonlint.com to find the line.

Every field explained

Field	What it does	Example	Needed?
`code`	Unique activity code. It's shown on the card, links the wiring sheet, and must match the firmware folder name.	`"CS-G08-A07"`	Yes
`name`	Activity name on the card and in the pop-ups.	`"Line Following Robot"`	Yes
`category`	Module filter and card icon. Use one of `Mechatronics`, `IoT/Automation`, `AI`, `Educator Resources`.	`"Mechatronics"`	Yes
`board`	Target board. Every Coggy is an ESP32, so keep `esp32`.	`"esp32"`	Yes
`version`	Version tag on the card. Raise it when you upload a new build.	`"1.0.0"`	Yes
`codeType`	`"table"` = Table wise code, a table picker and one build per table. `"universal"` = Universal code, no picker, and every table gets the `T1` build.	`"table"`	Yes
`description`	One or two sentences under the name on the card.	`"Two IR sensors..."`	Yes
`components`	Parts list for the card's "Needs" line. Use `[]` for none.	`["L298N", "IR sensor x2"]`	Yes
`parts`	Where the build is. Keep `{table}` in the path even for Universal activities, because the flasher fills it in. Keep `offset` at `"0x0"`.	see template	Yes
`wifiDetails`	`true` adds a "Wi-Fi credentials" button for educators, and `false` hides it.	`false`	Optional

Field	What it does	Example	Needed?
`ssid`	Wi-Fi name shown in that pop-up. Writing `{table}` puts in the table name in capitals, so `Home_MS_{table}` becomes `Home_MS_PI` on T1.	`"Home_MS_{table}"`	Optional
`password`	Wi-Fi password shown in that pop-up.	`"12345678"`	Optional
`ip`	Web address to open once connected.	`"192.168.4.1"`	Optional
`wifiNote`	A tip shown under the Wi-Fi details.	`"Connect your phone first."`	Optional
`soon`	`true` makes a greyed-out "Coming soon" placeholder that can't be flashed. Delete the line when the activity is ready.	`true`	Optional

Anyone can read the Wi-Fi password by opening the page source, so only use passwords meant for classroom kits.

Code naming: `CS-G` + two-digit class + `-A` + two-digit number, so `CS-G08-A07` is Class 8, activity 7. AI activities use `AI` instead of `A` (`CS-G05-AI03`), and Educator Resources use `CS-ER-A01`. Class folders are `grade-03` to `grade-09`, always two digits, and Educator Resources builds go in `firmware/resources/`.

Wiring sheet

The card's Wiring button shows the pins listed for that code under `window.COGGY_WIRING` in `index.html`. Search for `WIRING SHEETS` to find it. Without an entry, the pop-up shows a friendly "not added yet" message, so this step can wait.

Copy this block inside `window.COGGY_WIRING = { ... }`, under the right class comment:

```
  'CS-G08-A07': {
    pins: [
      { part: 'IR Sensor Left',  lead: 'OUT', pin: 'GPIO 34', kind: 'input'
},
      { part: 'IR Sensor Right', lead: 'OUT', pin: 'GPIO 35', kind: 'input'
},
      { part: 'Motor Driver',    lead: 'IN1', pin: 'GPIO 13', kind: 'motor'
},
      { part: 'Motor Driver',    lead: 'ENA', pin: 'GPIO 14', kind: 'pwm' },
      { part: 'IR Sensors',      lead: 'VCC', pin: '3V3',     kind: 'power'
},
      { part: 'IR Sensors',      lead: 'GND', pin: 'GND',     kind: 'ground'
},
    ],
    note: 'Calibrate the IR sensors on the black line first.',
  },
```

- **One line is one wire.** Copy a line to add a sensor, and delete it to remove one.
- `part` is the component as students know it. `lead` is the label printed on it, and you can leave it out. `pin` is the Coggy pin, such as `GPIO 25`, `3V3` or `GND`.
- `kind` sets the row's colour and icon: `input`, `analog`, `output`, `pwm`, `i2c`, `motor`, `power`, `ground`, `wireless` or `setup`.
- `note` is optional and shows as a tip under the list.
- This block is JavaScript, not JSON, so single quotes and a comma after every line are fine.

If the sketch was built with `CoggyWiring.h`, the flasher reads the pins from the `.bin` itself and skips this list.

Step by step

1. **Compile the sketch.** In the Arduino IDE, choose ESP32 Dev Module and use Sketch > Export Compiled Binary.
2. **Take the merged build.** The sketch folder gets a `build` folder. Use the file ending in `.ino.merged.bin`, which includes the bootloader, because the catalogue flashes at offset `0x0`. Rename it to `firmware.bin`.
3. **Place it.**
   - Table wise: compile once per table with that table's settings, then save each build to `firmware/grade-08/CS-G08-A07/T1/firmware.bin`, `T2/`, and so on.
   - Universal: compile once and save it to `firmware/grade-08/CS-G08-A07/T1/firmware.bin`.

4. **Add the catalogue entry.** Paste the template into the right class in `index.html`, change the values, and check the commas.
5. **Add the wiring sheet** if you have the pins ready.
6. **Test on your computer.** Open a terminal in the `coggy-flasher` folder, run `python -m http.server 8000`, and open `http://localhost:8000` in Chrome. Find the card, check its tag (Table wise or Universal), select it, and flash a Coggy.
7. **Upload.** Send the changed `index.html` and the new firmware folder to the server.
   - Vercel: push the changes to the GitHub repository Vercel deploys from, or drag the folder into a new deployment. The site updates in about a minute.
   - EC2: copy the files up with `scp` or WinSCP into `/var/www/coggy-flasher/`. No restart is needed.
8. **Check the live site.** Press Ctrl+Shift+R to skip the old cached page, then select the activity and confirm it says the program was found.

To update a build later, replace `firmware.bin`, raise `version` in the catalogue, and upload both.

Adding a new class

A class is one block in the `"grades": [ ... ]` list, holding its activities. Copy this, put a comma after the class before it, and make a matching `firmware/grade-10/` folder:

```
{
  "grade": 10,
  "title": "Advanced Robotics",
  "activities": [
    { ...activity template... }
  ],
  "color": "#D9467A"
}
```

`grade` is the class number shown as "Class 10", `title` is the subtitle on the class card, and `color` tints its cards and progress bar.

Common mistakes

What you see	Likely cause	Fix
The shelf shows 0 activities	A JSON mistake in the catalogue, usually a missing or extra comma	Check the block you just edited, or paste the catalogue into jsonlint.com

What you see	Likely cause	Fix
"Oh no, sorry!" pop-up on Flash	No `firmware.bin` at that path	Check the folder names, capital letters, and that the file is called `firmware.bin`
A Universal activity says the program is missing	The build isn't in the `T1` folder	Move it to `.../CODE/T1/firmware.bin`
The card shows the wrong icon or filter	`category` isn't one of the four names	Use `Mechatronics`, `IoT/Automation`, `AI` or `Educator Resources`
The Coggy doesn't start after flashing	The plain `.ino.bin` was used instead of the merged build	Re-export and use `.ino.merged.bin`
The live site still shows the old version	The browser cached the page	Press Ctrl+Shift+R

One to fix now: `CS-G08-A03` (Human Following Robot) has `"category": "universal"` and `"codeType": "table"`. It probably should be `"category": "Mechatronics"`, plus `"codeType": "universal"` if it uses one build for every table.